Guía práctica de APIs con Python

De una respuesta JSON a un análisis con Pandas

Este manual explica cómo pensar y trabajar con una API desde Python. El objetivo no es memorizar una receta, sino aprender a inspeccionar cada respuesta, reconocer dónde están los registros y decidir cómo convertirlos en una estructura útil para el análisis.

La idea central es sencilla: una API entrega una respuesta; JSON transporta la información; Python la convierte en diccionarios y listas; y Pandas resulta útil cuando queremos filtrar, limpiar, calcular métricas o guardar una tabla.

Ruta mental

URL → GET → respuesta HTTP → JSON → Python → DataFrame → análisis → CSV

Nivel: iniciación a análisis de datos con Python

Ejemplos: NASA Open APIs, NASA JPL Sentry y FRED S&P 500

Contenido

1. Qué es una API y qué ocurre en una llamada
2. El patrón mínimo con requests
3. Cómo inspeccionar una respuesta JSON
4. Cómo decidir qué son filas y columnas
5. Seis estructuras JSON frecuentes
6. Limpieza y análisis con Pandas
7. Ejemplos completos con NASA
8. Ejemplo financiero con el S&P 500
9. API y scraping web no son lo mismo
10. Paginación límites reintentos y seguridad
11. Diagnóstico de errores
12. Plantilla reutilizable y lista de comprobación

Puedes leer la guía en orden o utilizarla como referencia. Las secciones 3, 4 y 5 son las más importantes para aprender a moverte por respuestas que todavía no conoces.

1 Qué es una API y qué ocurre en una llamada

Una API es una interfaz diseñada para que un programa solicite datos o ejecute operaciones en otro sistema. En una API web, Python envía una petición HTTP a una URL y el servidor devuelve una respuesta. Esa respuesta puede contener JSON, XML, CSV, una imagen, un archivo o un mensaje de error.

Los componentes de una petición

Componente	Función	Ejemplo
URL	Identifica el servicio o recurso	https://api.nasa.gov/neo/rest/v1/feed
Método	Indica la operación	GET para consultar
Parámetros	Acotan o configuran la consulta	start_date, end_date
Autenticación	Identifica al cliente cuando es necesaria	api_key
Cabeceras	Añaden metadatos	Authorization, User-Agent
Cuerpo	Envía datos en POST o PUT	json={...}

GET se utiliza normalmente para consultar. POST suele crear o enviar información; PUT y PATCH actualizan; DELETE elimina. En análisis de datos trabajaremos sobre todo con GET.

API frente a página web

Una página web está pensada para una persona y devuelve HTML para que un navegador lo represente. Una API está pensada para programas y suele devolver datos estructurados. Si existe una API oficial, normalmente es preferible a extraer información del HTML porque la estructura es más estable, explícita y fácil de validar.

2 El patrón mínimo con requests

La mayoría de las consultas de este curso parten del mismo patrón:

Patrón mínimo

```
import requests
```

```
URL = "https://servicio.example/api/recurso"

params = {
    "parametro": "valor",
    "api_key": "TU_CLAVE"
}

response = requests.get(URL, params=params, timeout=30)
response.raise_for_status()
data = response.json()
```

Línea	Qué hace	Por qué importa
requests.get	Envía una petición GET	La consulta aún no es el JSON
raise_for_status	Detecta códigos HTTP de error	Evita analizar una respuesta fallida
response.json	Convierte JSON a objetos Python	Obtienes dict y list utilizables

Parámetros y URL final

El diccionario params no se envía como una tabla. Requests lo codifica en la URL. Esto evita concatenar texto manualmente y gestiona caracteres especiales.

```
params = {"date": "2025-10-28", "api_key": "DEMO_KEY"}
response = requests.get(URL, params=params)

print(response.url)
# https://.../recurso?date=2025-10-28&api_key=DEMO_KEY
```

El timeout limita cuánto tiempo esperará el programa. Sin él, una conexión problemática puede mantener el proceso bloqueado durante demasiado tiempo.

Autenticación

No todas las API utilizan el mismo sistema. Algunas no exigen autenticación; otras usan una API key como parámetro; y otras esperan un token en una cabecera. La documentación del endpoint indica cuál corresponde.

```
# API key como parámetro
params = {"api_key": API_KEY}

# Token en cabecera
headers = {"Authorization": f"Bearer {TOKEN}"}
response = requests.get(URL, headers=headers, timeout=30)
```

No publiques claves reales en notebooks o repositorios. Para proyectos personales, una opción sencilla es leerlas desde una variable de entorno.

```
import os

API_KEY = os.getenv("NOMBRE_API_KEY")
if not API_KEY:
    raise ValueError("Falta configurar NOMBRE_API_KEY")
```

3 Cómo inspeccionar una respuesta JSON

No conviertas a DataFrame a ciegas. Primero debes comprender la forma de la respuesta. No existe una clave principal universal y la primera clave de .keys() no tiene ningún significado especial.

Inspección básica

```
data = response.json()

print(type(data))

if isinstance(data, dict):
    print(data.keys())
```

```
    for clave, valor in data.items():
        print(clave, type(valor))

elif isinstance(data, list):
    print("Número de elementos:", len(data))
    print("Primer elemento:", data[0])
```

Para respuestas grandes, pprint ayuda a leer un ejemplo sin convertirlo todavía en tabla.

```
from pprint import pprint

pprint(data)
pprint(data["data"][0])  # Un registro de ejemplo
```

Correspondencia entre JSON y Python

JSON	Python	Uso habitual
object {}	dict	Conjunto de claves y valores
array []	list	Colección ordenada de elementos
string	str	Texto, fechas o números sin convertir
number	int o float	Cantidades numéricas
true false	True False	Valores booleanos
null	None	Dato ausente

Inspección recursiva opcional

Cuando el JSON tiene muchos niveles, esta función muestra la estructura sin imprimir todos los valores. Es una herramienta de exploración, no una transformación obligatoria.

```
def mostrar_estructura(objeto, nivel=0, max_elementos=3):
    sangria = "  " * nivel

    if isinstance(objeto, dict):
        print(f"{sangria}dict con {len(objeto)} claves")
        for clave, valor in list(objeto.items())[:max_elementos]:
            print(f"{sangria}- {clave}: {type(valor).__name__}")
            mostrar_estructura(valor, nivel + 1, max_elementos)

    elif isinstance(objeto, list):
        print(f"{sangria}list con {len(objeto)} elementos")
        if objeto:
            mostrar_estructura(objeto[0], nivel + 1, max_elementos)

mostrar_estructura(data)
```

4 Cómo decidir qué son filas y columnas

La decisión depende de la pregunta. Una fila suele representar una observación repetida: un asteroide, una fecha, una película, una ciudad o una operación. Las columnas describen atributos comparables de esas observaciones.

Ejemplo: si la pregunta es qué asteroides pasaron cerca de la Tierra, una fila debe representar un asteroide o una aproximación. Si la pregunta es cuántos asteroides hubo cada día, una fila puede representar una fecha.

Pregunta	Una fila representa	Columnas posibles
Qué objetos se acercaron	Un asteroide	id, nombre, diámetro
Cómo fue cada aproximación	Un acercamiento	fecha, velocidad, distancia

Pregunta	Una fila representa	Columnas posibles
Cuántos hubo cada día	Una fecha	fecha, cantidad, peligrosos
Cuál tiene más riesgo	Un objeto Sentry	nombre, ip, diámetro

No elijas una clave porque aparezca primero. Identifica dónde están los registros repetidos y comprueba que cada elemento tenga una estructura parecida. La documentación y la inspección del primer elemento suelen bastar.

5 Seis estructuras JSON frecuentes

Caso 1 lista de diccionarios

```
data = [
    {"nombre": "A", "diametro": 20},
    {"nombre": "B", "diametro": 35}
]

df = pd.DataFrame(data)
```

Cada elemento de la lista es una fila y las claves de cada diccionario son columnas.

Caso 2 diccionario que contiene la lista

```
data = {
    "count": 2,
    "resultados": [
        {"nombre": "A", "diametro": 20},
        {"nombre": "B", "diametro": 35}
    ]
}

df = pd.DataFrame(data["resultados"])
```

El contador es metadato. La lista resultados contiene los registros que queremos tabular.

Caso 3 diccionario de listas

```
data = {
    "nombre": ["A", "B"],
    "diametro": [20, 35]
}

df = pd.DataFrame(data)
```

Las claves ya son columnas y las listas tienen el mismo número de posiciones.

Caso 4 un solo registro

```
data = {"nombre": "A", "diametro": 20}

df = pd.DataFrame([data])
```

La lista exterior indica que el diccionario es una única fila.

Caso 5 diccionario de diccionarios

```
data = {
    "asteroide_1": {"nombre": "A", "diametro": 20},
    "asteroide_2": {"nombre": "B", "diametro": 35}
}

df = pd.DataFrame.from_dict(data, orient="index")
df = df.reset_index(names="identificador")
```

orient='index' hace que cada diccionario interior sea una fila.

Caso 6 listas agrupadas por otra clave

```
data = {
    "2026-11-01": [{"nombre": "A"}, {"nombre": "B"}],
    "2026-11-02": [{"nombre": "C"}]
}

filas = []

for fecha, registros in data.items():
    for registro in registros:
        filas.append({
            "fecha": fecha,
            "nombre": registro["nombre"]
        })

df = pd.DataFrame(filas)
```

Aquí es necesario aplanar las listas. Además conservamos la fecha, porque al extraer los elementos perderíamos el grupo al que pertenecían.

Cuándo usar json_normalize

pd.json_normalize intenta expandir diccionarios anidados. Resulta práctico para una primera exploración, aunque puede producir nombres de columna largos y no siempre aplana listas internas.

```
registros = data["resultados"]
df = pd.json_normalize(registros)

print(df.columns.tolist())
```

6 Limpieza y análisis con Pandas

Crear el DataFrame no garantiza que los tipos sean correctos. Muchas API envían números y fechas como texto. Antes de calcular métricas, revisa la estructura.

Diagnóstico inicial

```
df.head()
df.info()
df.shape
df.columns.tolist()
df.isna().sum()
```

Conversión de tipos

```
df["ip"] = pd.to_numeric(df["ip"], errors="coerce")
df["fecha"] = pd.to_datetime(df["fecha"], errors="coerce")
df["peligroso"] = df["peligroso"].astype("boolean")
```

errors='coerce' convierte valores no válidos en NaN o NaT. Después puedes decidir si eliminarlos, rellenarlos o investigarlos.

Operaciones habituales

```
df["ip"].max()
df["diametro"].mean()
df.sort_values("ip", ascending=False).head(10)
df[df["peligroso"] == True]
df.groupby("fecha").size()
df.groupby("fecha")["diametro"].mean()
df.describe(include="all")
```

Guardado y reutilización

```
df.to_csv("resultado.csv", index=False, encoding="utf-8-sig")

# Recuperar los datos sin repetir la llamada
df = pd.read_csv("resultado.csv")
```

Guardar evita consumir de nuevo el límite de peticiones y conserva una instantánea reproducible. La codificación utf-8-sig facilita abrir textos con tildes en Excel para Windows.

7 Ejemplos completos con NASA

APOD foto astronómica del día

```
import requests

URL_APOD = "https://api.nasa.gov/planetary/apod"

params = {
    "date": "2025-10-28",
    "api_key": "DEMO_KEY"
}

response = requests.get(URL_APOD, params=params, timeout=30)
response.raise_for_status()
data = response.json()

print(data["title"])
print(data["explanation"])

imagen = requests.get(data["url"], timeout=30)
imagen.raise_for_status()

with open("nasa_cumple.jpg", "wb") as archivo:
    archivo.write(imagen.content)
```

La primera respuesta es JSON y contiene metadatos. La segunda respuesta es binaria porque descarga una imagen. No todas las respuestas de una API se convierten a DataFrame.

NeoWs asteroides cercanos

```
import pandas as pd
from datetime import date, timedelta

URL_NEO = "https://api.nasa.gov/neo/rest/v1/feed"

params = {
    "start_date": str(date.today() - timedelta(days=6)),
    "end_date": str(date.today()),
    "api_key": "DEMO_KEY"
}

response = requests.get(URL_NEO, params=params, timeout=30)
response.raise_for_status()
data = response.json()

filas = []

for fecha, lista_asteroides in data["near_earth_objects"].items():
    for asteroide in lista_asteroides:
        aproximacion = asteroide["close_approach_data"][0]
        diametro = asteroide["estimated_diameter"]["kilometers"]

        filas.append({
            "fecha": fecha,
            "id": asteroide["id"],
            "nombre": asteroide["name"],
            "diametro_min_km": diametro["estimated_diameter_min"],
            "diametro_max_km": diametro["estimated_diameter_max"],
            "peligroso": asteroide["is_potentially_hazardous_asteroid"],
            "velocidad_km_s": float(
                aproximacion["relative_velocity"]["kilometers_per_second"]
            ),
            "distancia_km": float(
                aproximacion["miss_distance"]["kilometers"]
            )
        })
```

```
asteroides = pd.DataFrame(filas)
asteroides.to_csv("asteroids.csv", index=False)
```

La parte decisiva es reconocer que near_earth_objects es un diccionario de fechas y que cada fecha contiene una lista. El bucle aplana esa jerarquía y conserva la fecha como columna.

Sentry objetos con riesgo de impacto

```
URL_SENTRY = "https://ssd-api.jpl.nasa.gov/sentry.api"

response = requests.get(URL_SENTRY, timeout=30)
response.raise_for_status()
data = response.json()

print(data.keys())
# dict_keys(['signature', 'count', 'data'])

sentry = pd.DataFrame(data["data"])
sentry["ip"] = pd.to_numeric(sentry["ip"], errors="coerce")

mayor_probabilidad = sentry.loc[[sentry["ip"].idxmax()]]
sentry.to_csv("sentry.csv", index=False)

mayor_probabilidad
```

En Sentry, la lista de registros está directamente en data. ip representa la probabilidad acumulada de impacto y llega como texto, por lo que se convierte antes de aplicar idxmax.

8 Ejemplo financiero con el S&P 500

Este ejemplo utiliza la API oficial de FRED, mantenida por el Banco de la Reserva Federal de St. Louis. La serie SP500 contiene valores diarios de cierre del índice. FRED exige una API key y puede devolver JSON cuando file_type='json'.

Objetivo del análisis

- Descargar observaciones del S&P 500 desde una fecha determinada.
- Convertir la lista observations en un DataFrame.
- Limpiar los días sin valor, representados con un punto.
- Calcular rentabilidad diaria, media móvil y métricas descriptivas.
- Guardar el resultado en CSV.

1 Llamada e inspección

```
import os
import pandas as pd
import requests

URL_FRED = "https://api.stlouisfed.org/fred/series/observations"
FRED_API_KEY = os.getenv("FRED_API_KEY")

if not FRED_API_KEY:
    raise ValueError("Configura la variable de entorno FRED_API_KEY")

params = {
    "series_id": "SP500",
    "api_key": FRED_API_KEY,
    "file_type": "json",
    "observation_start": "2025-01-01",
    "sort_order": "asc"
}

response = requests.get(URL_FRED, params=params, timeout=30)
response.raise_for_status()
data = response.json()

print(data.keys())
print(type(data["observations"]))
print(data["observations"][0])
```

La respuesta contiene metadatos y una lista observations. Esa lista es la unidad tabular: cada elemento representa una fecha y un valor del índice.

2 Conversión y limpieza

```
sp500 = pd.DataFrame(data["observations"])

sp500 = sp500[["date", "value"]].copy()
sp500["date"] = pd.to_datetime(sp500["date"])
sp500["value"] = pd.to_numeric(sp500["value"], errors="coerce")

# El punto usado por FRED para un día sin observación se convierte en NaN.
sp500 = sp500.dropna(subset=["value"])
sp500 = sp500.sort_values("date").reset_index(drop=True)
```

3 Métricas y guardado

```
sp500["rentabilidad_diaria_pct"] = sp500["value"].pct_change() * 100
sp500["media_movil_20"] = sp500["value"].rolling(20).mean()

print("Último cierre:", sp500.iloc[-1]["value"])
print("Máximo del periodo:", sp500["value"].max())
print("Mínimo del periodo:", sp500["value"].min())
print("Rentabilidad media diaria:", sp500["rentabilidad_diaria_pct"].mean())

fila_maximo = sp500.loc[[sp500["value"].idxmax()]]
print(fila_maximo)

sp500.to_csv("sp500_fred.csv", index=False, encoding="utf-8-sig")
```

pct_change compara cada valor con el anterior. rolling(20).mean calcula una media móvil de veinte observaciones. Estas operaciones solo funcionan de forma fiable después de convertir value a número y ordenar por fecha.

La serie SP500 de FRED es un índice de precios, no un índice de rentabilidad total: no incorpora dividendos. FRED indica además que la disponibilidad histórica de esta serie está sujeta al acuerdo con S&P Dow Jones Indices. Revisa las notas y condiciones antes de redistribuir los datos.

9 API y scraping web no son lo mismo

Aspecto	API	Scraping web
Fuente	Endpoint diseñado para programas	HTML diseñado para personas
Estructura	JSON XML o CSV	Etiquetas HTML
Estabilidad	Contrato documentado	Puede cambiar con el diseño
Acceso	Clave límites y parámetros	Robots términos y carga del sitio
Herramientas	requests	BeautifulSoup o read_html

El ejemplo de FRED es extracción mediante API, no scraping. El scraping se utiliza cuando la información solo aparece en una página y no existe un endpoint adecuado. Antes de hacerlo, revisa las condiciones de uso, robots.txt, derechos sobre los datos y frecuencia de acceso.

Ejemplo educativo de una tabla HTML

La página de Wikipedia con las empresas del S&P 500 contiene una tabla HTML. Pandas puede leer tablas visibles con read_html. Este método depende de que la página mantenga su estructura.

```
import pandas as pd

URL_COMPONENTES = (
    "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
)
```

```
tablas = pd.read_html(URL_COMPONENTES)
componentes = tablas[0]

print(componentes.columns)
print(componentes.head())

componentes.to_csv(
    "componentes_sp500.csv",
    index=False,
    encoding="utf-8-sig"
)
```

Aquí también hay una decisión: tablas[0] significa que seleccionamos la primera tabla encontrada. En un proyecto real conviene comprobar sus columnas y no asumir eternamente que seguirá en esa posición.

Inspección de tablas encontradas

```
for numero, tabla in enumerate(tablas):
    print(numero, tabla.shape, tabla.columns.tolist())
```

10 Paginación límites reintentos y seguridad

Paginación

Muchas API no devuelven todos los registros en una sola respuesta. Utilizan page, offset, limit o un enlace next. Debes repetir peticiones hasta terminar y acumular los registros.

```
todos = []
pagina = 1

while True:
    params = {"page": pagina, "limit": 100}
    response = requests.get(URL, params=params, timeout=30)
    response.raise_for_status()
    data = response.json()

    registros = data["results"]
    todos.extend(registros)

    if not data.get("next"):
        break

    pagina += 1

df = pd.DataFrame(todos)
```

Límites de peticiones

Las API pueden limitar peticiones por minuto, hora o día. Un 429 significa normalmente demasiadas solicitudes. Guarda respuestas, evita repetir consultas innecesarias y respeta Retry-After cuando exista.

Sesiones y reintentos

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()

retry = Retry(
    total=3,
    backoff_factor=1,
    status_forcelist=[429, 500, 502, 503, 504],
    allowed_methods=["GET"]
)

session.mount("https://", HTTPAdapter(max_retries=retry))

response = session.get(URL, params=params, timeout=30)
response.raise_for_status()
```

Los reintentos son útiles para fallos temporales, no para ocultar una URL o una clave incorrectas. El backoff espera progresivamente entre intentos.

Validación antes de analizar

```
data = response.json()

if "data" not in data:
    raise ValueError(f"Estructura inesperada: {data.keys()}")

if not isinstance(data["data"], list):
    raise TypeError("Se esperaba una lista en data['data']")

if not data["data"]:
    print("La consulta no devolvió registros")
```

Seguridad básica

- No incluyas API keys reales en Git, capturas o documentos públicos.
- Usa HTTPS para evitar transmitir datos en texto plano.
- Concede a cada token los permisos mínimos necesarios.
- No imprimas cabeceras de autorización en logs compartidos.
- Revoca y sustituye una clave si se ha publicado accidentalmente.
- Lee las condiciones de uso y las restricciones de redistribución.

11 Diagnóstico de errores

Código o error	Significado habitual	Qué revisar
400	Petición incorrecta	Nombres, formatos y combinaciones de parámetros
401	No autorizado	Clave ausente, incorrecta o endpoint equivocado
403	Acceso prohibido	Permisos, cuota o restricciones
404	Recurso no encontrado	Ruta y versión del endpoint
429	Demasiadas peticiones	Límites, Retry-After y caché
500 503	Error temporal del servidor	Reintentar con espera
JSONDecodeError	La respuesta no es JSON válido	Content-Type y response.text
KeyError	La clave no existe	data.keys y estructura real
TypeError	El tipo no es el esperado	type de cada nivel

Bloque de diagnóstico

```
response = requests.get(URL, params=params, timeout=30)

print("Estado:", response.status_code)
print("URL final:", response.url)
print("Content-Type:", response.headers.get("Content-Type"))
print("Primeros caracteres:", response.text[:300])

response.raise_for_status()
```

No compartas response.url si contiene una API key real. En ese caso, oculta el valor antes de copiar el mensaje de error.

Secuencia recomendada cuando algo falla

13. Lee el código HTTP y el mensaje devuelto.

14. Imprime la URL final sin revelar credenciales.
15. Compara endpoint y parámetros con la documentación actual.
16. Comprueba el formato de fechas y valores booleanos.
17. Inspecciona type, keys y un registro real.
18. Solo después modifica la transformación a DataFrame.

12 Plantilla reutilizable y lista de comprobación

Plantilla para una nueva API

```
import os
import pandas as pd
import requests

# 1. Configuración
URL = "https://servicio.example/api/recurso"
API_KEY = os.getenv("SERVICIO_API_KEY")

params = {
    "parametro": "valor",
    "api_key": API_KEY
}

# 2. Petición
response = requests.get(URL, params=params, timeout=30)
response.raise_for_status()
data = response.json()

# 3. Inspección
print(type(data))
if isinstance(data, dict):
    print(data.keys())

# 4. Selección de registros
registros = data["CAMBIAR_POR_CLAVE_REAL"]
print(type(registros), len(registros))
print(registros[0])

# 5. DataFrame y tipos
df = pd.DataFrame(registros)
# df["numero"] = pd.to_numeric(df["numero"], errors="coerce")
# df["fecha"] = pd.to_datetime(df["fecha"], errors="coerce")

# 6. Validación y análisis
print(df.shape)
print(df.info())
print(df.head())

# 7. Persistencia
df.to_csv("resultado.csv", index=False, encoding="utf-8-sig")
```

Lista de comprobación

☐ Sé qué pregunta quiero responder.

☐ He localizado el endpoint correcto y su documentación.

☐ Sé si requiere clave, token o ninguna autenticación.

☐ He definido timeout y comprobado el código HTTP.

☐ He inspeccionado type, keys y al menos un registro.

☐ He decidido qué representa una fila.

☐ He conservado las claves de agrupación importantes, como fecha o categoría.

☐ He convertido números y fechas antes de calcular métricas.

☐ He revisado valores ausentes y duplicados.

☐ He guardado una copia local si la consulta es costosa o limitada.

☐ No he expuesto credenciales.

☐ He respetado las condiciones de uso y límites del servicio.

Qué debes recordar

No necesitas memorizar todas las estructuras. Necesitas saber inspeccionar una respuesta, reconocer el patrón de registros y transformar solo lo necesario para tu pregunta. A veces el DataFrame se crea en una línea; otras veces hay que aplanar varios niveles. Esa decisión forma parte del análisis, no es un error.

JSON no es un obstáculo: después de response.json() ya tienes diccionarios y listas de Python. Puedes trabajar directamente con ellos. Pandas entra cuando la información tiene sentido tabular y quieres comparar observaciones, limpiar tipos, calcular métricas o exportar resultados.

Fuentes y documentación

- NASA Open APIs: https://api.nasa.gov/
- NASA JPL Sentry API: https://ssd-api.jpl.nasa.gov/doc/sentry.html
- FRED API series observations: https://fred.stlouisfed.org/docs/api/fred/series_observations.html
- FRED S&P 500 series SP500: https://fred.stlouisfed.org/series/SP500
- Requests documentation: https://requests.readthedocs.io/
- Pandas DataFrame: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html
- Pandas json_normalize: https://pandas.pydata.org/docs/reference/api/pandas.json_normalize.html

La documentación del endpoint es la referencia definitiva para parámetros, límites, estructura de la respuesta y condiciones de uso. Las API pueden cambiar; revisa la versión vigente cuando un ejemplo deje de funcionar.